



LUBELSKA AKADEMIA WSEI
WYDZIAŁ TRANSPORTU I INFORMATYKI
KIERUNEK: INFORMATYKA
Inżynieria Oprogramowania i Technologie Mobilne

Michał Białek

NR ALBUMU: 41633

Projekt inżynierski

PROJEKT I IMPLEMENTACJA INTELIGENTNEGO SYSTEMU STEROWANIA MIKROKLIMATEM SZKLARNI Z WYKORZYSTANIEM CZUJNIKÓW IOT I ANALIZY DANYCH

Design and Implementation of an Intelligent Greenhouse Microclimate Control System Using IoT Sensors and Data Analysis

Opiekun projektu: dr inż Tomasz Giżewski

Lublin 2026

SPIS TREŚCI

Wstęp	3
ROZDZIAŁ 1. Analiza problemu i wymagań systemu.....	5
1.1 Cel pracy	5
1.2 Analiza istniejących rozwiązań	6
1.3 Określenie środowiska działania systemu	8
ROZDZIAŁ 2. Projekt systemu	11
2.1 Wymagania funkcjonalne.....	11
2.2 Wymagania нефункционалне	12
2.2.1 Przypadki użycia	13
2.3 Architektura systemu.....	13
2.4 Przepływ danych w systemie	16
2.4.1 Diagramy sekwencji	18
2.5 Algorytm sterowania mikroklimatem.....	19
2.6 Pętla sprzężenia zwrotnego	20
ROZDZIAŁ 3. Implementacja systemu	23
3.1 Struktura projektu i technologie.....	23
3.2 Warstwa urządzenia (ESP32)	24
3.3 Baza danych PostgreSQL	25
3.3.1 Tabela <code>devices</code>	26
3.3.2 Tabela <code>measurements</code>	26
3.3.3 Tabela <code>control_actions</code>	26
3.3.4 Tabela <code>llm_analyses</code>	26
3.4 Proces uruchamiania	28
3.5 Warstwa komunikacyjna (MQTT)	29

3.6 Bezpieczeństwo i autoryzacja	30
ROZDZIAŁ 4. Testy i wyniki.....	31
4.1 Scenariusze testowe.....	31
4.1.1 Tabela przypadków testowych	32
4.2 Testy wydajności	35
4.3 Analiza wyników.....	35
ROZDZIAŁ 5. Wnioski i perspektywy rozwoju.....	39
5.1 Podsumowanie	39
5.2 Ograniczenia systemu	39
5.3 Możliwości rozbudowy.....	40

WSTĘP

Szybki rozwój technologii Internetu Rzeczy (IoT) i sztucznej inteligencji (AI) prowadzi do nowych perspektyw automatyzacji procesów i dostępu do danych w czasie rzeczywistym. Jednym z konkretnych obszarów intensywnego wykorzystania tych technologii jest rolnictwo precyzyjne, z naciskiem na optymalizację warunków poprzez zastosowanie dostępnych zasobów. Integracja czujników środowiskowych i systemów analitycznych w uprawach szklarniowych umożliwia ciągłe monitorowanie mikroklimatu i jego automatyczną regulację. Takie rozwiązania pozwalają na większe plony, znacznie mniejsze zużycie wody i energii oraz mniejszą ingerencję człowieka w produkcję. Celem tej pracy jest opracowanie systemu wspierającego zarządzanie mikroklimatem szklarniowym w oparciu o technologie IoT i elementy sztucznej inteligencji. Pomiar, analiza i automatyzacja parametrów środowiskowych w czasie rzeczywistym stały się podstawą systemu. Na poziomie sprzętowym wykorzystywany jest mikrokontroler ESP32 – urządzenie, które zbiera dane z czujników i następnie przesyła je do lokalnego serwera opartego na Raspberry Pi. Oprogramowanie serwera w Pythonie zajmuje się przetwarzaniem danych wraz z podejmowaniem decyzji. Ponadto, te algorytmy analityczne i niektóre gałęzie sztucznej inteligencji pomagają zarówno w reagowaniu na nowe zmiany w środowisku, jak i w przewidywaniu oraz stopniowej aktualizacji jego wydajności. Obejmuje to projektowanie architektury sprzętowej i programowej systemu, wdrożenie systemu oraz ocenę jego skuteczności na podstawie przeprowadzonych testów.

ROZDZIAŁ 1.

ANALIZA PROBLEMU I WYMAGAŃ SYSTEMU

1.1. CEL PRACY

Celem niniejszej pracy jest stworzenie oraz walidacja inteligentnego systemu kontroli mikroklimatu szklarni z wykorzystaniem technologii Internetu Rzeczy, modeli analizy danych oraz podejść sztucznej inteligencji. System ma zapewniać autonomiczne monitorowanie i regulację kluczowych parametrów środowiska, takich jak temperatura oraz wilgotność powietrza i gleby. Istotnym założeniem jest zwiększenie efektywności procesu uprawy poprzez racjonalizację wykorzystania zasobów (wody, energii) oraz stabilizację warunków mikroklimatycznych, co pozwoli na ograniczenie wpływu czynnika ludzkiego i zminimalizowanie ryzyka błędów operacyjnych.

Realizacja głównego celu wymaga zaprojektowania kompletnej architektury systemu, integrującej warstwę czujników, elementy wykonawcze oraz moduły komunikacyjne, a także implementacji mechanizmów przetwarzania danych w czasie rzeczywistym. W ramach pracy opracowany zostanie moduł decyzyjny w języku Python, odpowiedzialny za autonomiczne sterowanie systemami nawadniania, wentylacji i ogrzewania. System będzie również umożliwiał prezentację zgromadzonych danych pomiarowych i wyników analiz w formie czytelnej dla użytkownika.

Kluczowym aspektem projektowanego rozwiązania jest zapewnienie możliwości pracy systemu w trybie offline, co uniezależnia jego działanie od ciągłego dostępu do sieci Internet. Taka funkcjonalność znacząco zwiększa niezawodność rozwiązania oraz jego odporność na awarie infrastruktury sieciowej, gwarantując ciągłość krytycznych procesów sterowania niezależnie od dostępności usług zewnętrznych.

1.2. ANALIZA ISTNIEJĄCYCH ROZWIĄZAŃ

Istnieje wiele systemów wspomagających zarządzanie uprawami, które wykorzystują technologie Internetu Rzeczy oraz metody analizy danych. Do najbardziej rozpoznawalnych rozwiązań należą AgroSense[1], Farm Smart[2] oraz Climate FieldView[3]. Systemy te oferują zaawansowane funkcje monitorowania parametrów środowiskowych, analizy danych historycznych oraz generowania raportów wspierających podejmowanie decyzji agronomicznych.

Pomimo szerokiego zakresu funkcjonalności, komercyjne rozwiązania tego typu posiadają istotne ograniczenia. W większości przypadków wymagają one stałego dostępu do Internetu oraz korzystania z infrastruktury chmurowej, co uzależnia użytkownika od zewnętrznych dostawców usług. Dodatkowo dane pomiarowe są zazwyczaj przechowywane na serwerach producenta, co ogranicza kontrolę użytkownika nad ich przetwarzaniem i archiwizacją. Wysokie koszty wdrożenia oraz opłaty abonamentowe, a także zamknięta architektura systemów, znacząco utrudniają ich modyfikację i rozbudowę.

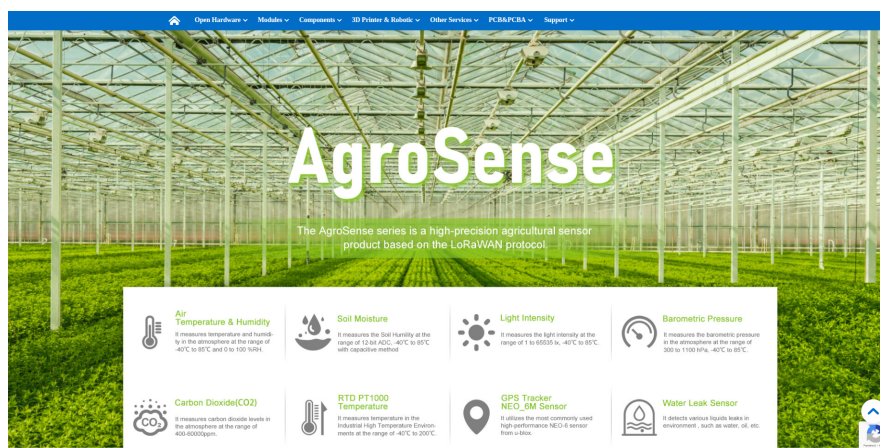
Alternatywą dla rozwiązań komercyjnych są projekty open-source, najczęściej oparte na mikrokontrolerach **Arduino** oraz **ESP32**. Charakteryzują się one niskimi kosztami wdrożenia oraz dużą elastycznością w zakresie konfiguracji sprzętowej i programowej. Ich istotnym ograniczeniem jest jednak zazwyczaj brak zaawansowanych mechanizmów analizy danych i funkcji predykcyjnych, a także niski stopień automatyzacji procesów decyzyjnych.

System zaproponowany w niniejszej pracy został zaprojektowany jako lokalna, otwarta platforma kontroli mikroklimatu szklarni, stanowiąca kompromis pomiędzy funkcjonalnością rozwiązań komercyjnych a elastycznością projektów open-source. Umożliwia on pracę w trybie offline, zapewnia pełną kontrolę użytkownika nad danymi pomiarowymi oraz pozwala na łatwą rozbudowę zarówno warstwy sprzętowej, jak i programowej przy zachowaniu niskich kosztów wdrożenia.

Dodatkowo proponowane rozwiązanie integruje moduł analizy danych wspomagany algorytmami sztucznej inteligencji, co umożliwia nie tylko reagowanie na bieżące zmiany parametrów środowiskowych, lecz także predykcję przyszłych warunków mikroklimatycznych, a tym samym bardziej efektywne sterowanie procesem uprawy.

A) AGROSENSE

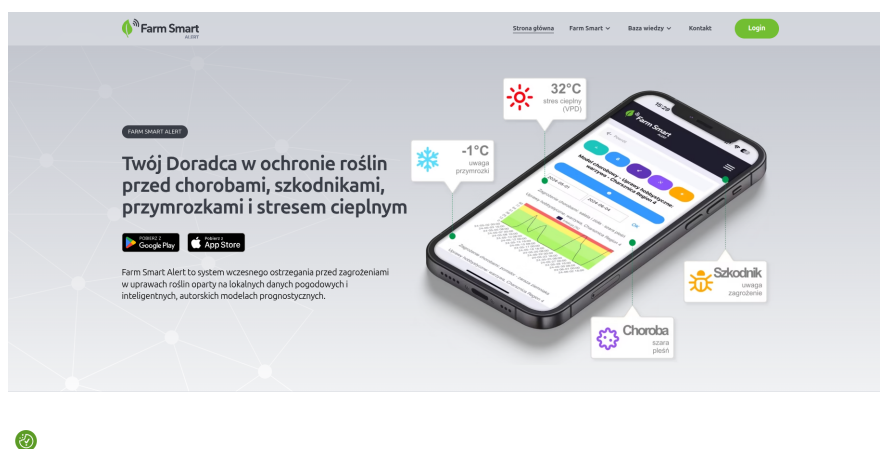
AgroSense jest komercyjnym systemem wspomagającym zarządzanie uprawami, opartym na technologiach Internetu Rzeczy oraz analizie danych środowiskowych. System umożliwia monitorowanie parametrów mikroklimatu, gromadzenie danych historycznych oraz ich analizę w celu wsparcia decyzji agronomicznych. Rozwiązanie to działa w oparciu o infrastrukturę chmurową i jest dostępne w modelu subskrypcyjnym.



Rys. 1. Strona domowa systemu AgroSense

B) FARM SMART

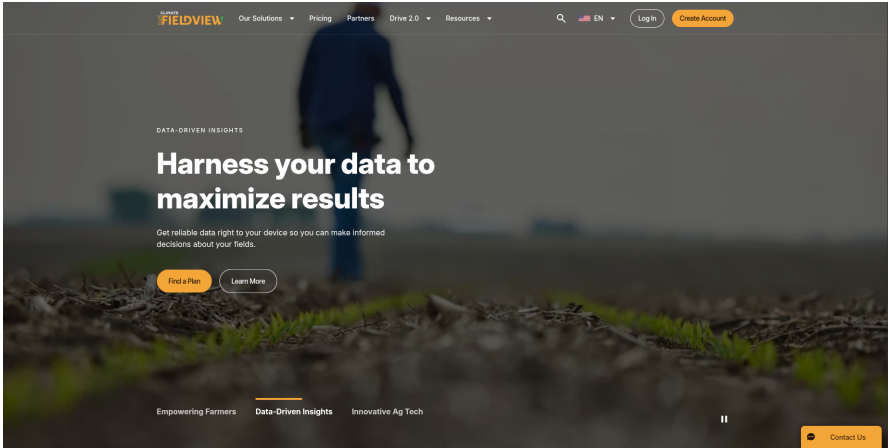
Farm Smart to platforma przeznaczona do monitorowania i analizy warunków środowiskowych w uprawach rolniczych i szklarniowych. System oferuje integrację z różnymi czujnikami oraz narzędzia do wizualizacji danych i generowania raportów. Podobnie jak inne rozwiązania komercyjne, Farm Smart wymaga stałego dostępu do Internetu oraz korzystania z usług chmurowych.



Rys. 2. Strona domowa systemu Farm Smart

C) CLIMATE FIELDVIEW

Climate FieldView jest zaawansowaną platformą analityczną przeznaczoną do zarządzania uprawami, skupiającą się na analizie danych środowiskowych oraz predykcji warunków wzrostu. System oferuje rozbudowane narzędzia analityczne oraz integrację z innymi usługami firmy Bayer. Ze względu na zamkniętą architekturę oraz model chmurowy użytkownik ma ograniczoną możliwość modyfikacji i dostosowania systemu do własnych potrzeb.



Rys. 3. Strona domowa systemu Climate FieldView

Tab. 1.1. Porównanie analizowanych rozwiązań komercyjnych z projektowanym systemem.

Cecha	AgroSense	Farm Smart	Climate FieldView	Proponowany System
Tryb pracy	Online (Chmura)	Online (Chmura)	Online (Chmura)	Offline / Lokalny
Model kosztów	Subskrypcja	Subskrypcja	Wysoki (Licencje)	Niski (Tylko sprzęt)
Lokalizacja danych	Serwer producenta	Serwer producenta	Chmura producenta	Własna baza SQL
Architektura	Zamknięta	Zamknięta	Zamknięta	Otwarta (Open Source)
Możliwość modyfikacji	Brak	Ograniczona	Brak	Pełna
Analiza danych	Algorytmy chmurowe	Statystyki	Big Data	Lokalna + LLM

1.3. OKREŚLENIE ŚRODOWISKA DZIAŁANIA SYSTEMU

Opracowany system został zaprojektowany z myślą o pracy w warunkach szklarniowych. Środowisko to umożliwia kontrolę mikroklimatu, jednak jednocześnie wymaga ciągłego monitorowania oraz automatycznej regulacji kluczowych parametrów środowiskowych. Szklarnia stanowi zamknięty ekosystem, w którym zachodzą procesy wymiany ciepła, parowania oraz fotosyntezy, dlatego system sterowania musi dynamicznie reagować na stale zmieniające się warunki wewnętrzne i zewnętrzne.

Podstawowym zadaniem systemu jest utrzymanie optymalnych warunków wzrostu roślin niezależnie od wpływu czynników zewnętrznych, takich jak temperatura otoczenia czy natężenie światła słonecznego. Wykorzystanie technologii Internetu Rzeczy (IoT) umożliwia automatyczne pozyskiwanie danych z czujników, ich analizę oraz sterowanie urządzeniami wykonawczymi w czasie rzeczywistym.

Do głównych parametrów środowiska szklarniowego należą:

- **temperatura i wilgotność powietrza**, wpływające na procesy metaboliczne oraz intensywność transpiracji roślin,
- **natężenie oświetlenia**, niezbędne do prawidłowego przebiegu procesu fotosyntezy,

- **stężenie CO₂**, istotne dla efektywności procesu fotosyntezy,
- **wilgotność gleby, pH oraz przewodność elektryczna (EC)**, determinujące gospodarkę wodno-mineralną roślin.

W celu skutecznej regulacji mikroklimatu szklarni system musi uwzględniać zarówno czynniki zewnętrzne, takie jak natężenie światła i temperatura otoczenia, jak i czynniki wewnętrzne, w tym liczbę roślin oraz etap ich wzrostu. Kluczowym aspektem jest zapewnienie wiarygodności pomiarów, co wymaga odpowiedniego rozmieszczenia i kalibracji czujników, a także odpornej na zakłócenia komunikacji bezprzewodowej w warunkach podwyższonej wilgotności.

Dzięki możliwości pełnej kontroli warunków środowiskowych, wysokiej powtarzalności eksperymentów oraz jednoznacznej ocenie skuteczności działania, środowisko szklarniowe stanowi idealny poligon doświadczalny dla opracowanego systemu.

ROZDZIAŁ 2.

PROJEKT SYSTEMU

Projektowany system stanowi modułowy system kontroli mikroklimatu w szklarni, który może być dalej rozwijany oraz stosowany w różnych warunkach uprawy przy minimalnych zmianach parametrów konfiguracyjnych. Architektura systemu ma charakter trójwarstwowy i obejmuje warstwę sprzętową, zawierającą czujniki oraz elementy wykonawcze, warstwę komunikacyjną, odpowiedzialną za transmisję danych pomiędzy urządzeniami, a także warstwę aplikacyjną, której zadaniem jest przetwarzanie, analiza oraz prezentacja danych użytkownikowi.

Zastosowany wzorzec architektury warstwowej jest zgodny z powszechnie stosowanymi modelami architektury systemów Internetu Rzeczy (IoT), w których warstwy fizyczna, sieciowa oraz aplikacyjna funkcjonują jako logicznie odrębne komponenty. Takie podejście umożliwia łatwą rozbudowę, adaptację systemu do nowych warunków oraz niezależny rozwój poszczególnych elementów architektury [4].

2.1. WYMAGANIA FUNKCJONALNE

Na podstawie przeprowadzonej analizy problemu zdefiniowano zestaw wymagań funkcjonalnych, które określają zdolności systemu do wykonywania zadań niezbędnych do realizacji celu pracy. Wymagania te stanowią punkt odniesienia dla dalszych decyzji projektowych, w szczególności w zakresie doboru elementów sprzętowych, protokołów komunikacji oraz sposobu przetwarzania i prezentacji danych.

System musi realizować następujące funkcje:

- **Akwizycja danych środowiskowych:** System musi cyklicznie pobierać dane z czujników mierzących temperaturę powietrza, wilgotność powietrza, wilgotność gleby, temperaturę

gleby, natężenie oświetlenia, stężenie CO₂ oraz parametry roztworu (pH, EC).

- **Bezprzewodowa transmisja danych:** Dane pomiarowe muszą być przesyłane z węzłów pomiarowych do serwera centralnego w czasie rzeczywistym z wykorzystaniem protokołu MQTT.
- **Archiwizacja danych:** Wszystkie odebrane pomiary muszą być trwale zapisywane w relacyjnej bazie danych z oznaczeniem czasowym (timestamp) oraz identyfikatorem urządzenia.
- **Automatyczne sterowanie:** System musi autonomicznie podejmować decyzje o uruchomieniu urządzeń wykonawczych (wentylatory, pompy, oświetlenie) w oparciu o zdefiniowane progi wartości parametrów (logika progowa).
- **Integracja z modelem LLM:** System powinien umożliwiać przesyłanie zagregowanych danych do zewnętrznego API modelu językowego w celu uzyskania opisowej analizy warunków.
- **Wizualizacja stanu:** System musi udostępniać interfejs umożliwiający podgląd aktualnych odczytów z czujników.

2.2. WYMAGANIA NIEFUNKCJONALNE

Wymagania niefunkcjonalne określają atrybuty jakościowe systemu oraz ograniczenia, w jakich musi on pracować. W praktyce wpływają one na decyzje architektoniczne (np. dobór lokalnej infrastruktury obliczeniowej) oraz na sposób implementacji komunikacji i obsługi błędów.

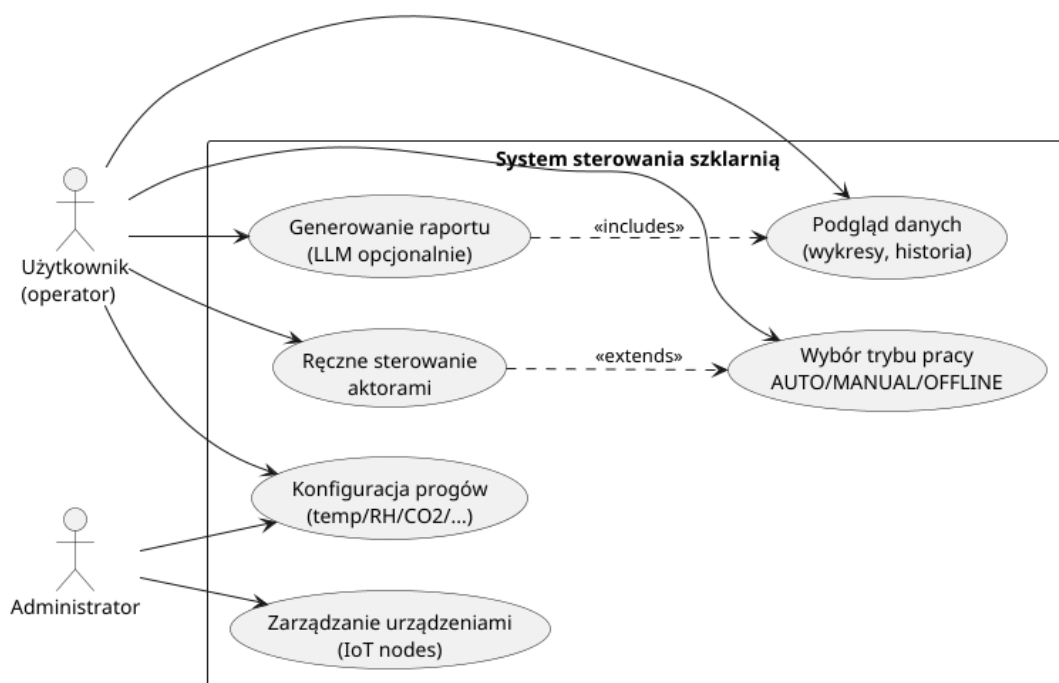
Zdefiniowano następujące wymagania:

- **Niezależność od usług chmurowych (Tryb Offline):** System musi zachować pełną funkcjonalność w zakresie sterowania i gromadzenia danych nawet w przypadku braku dostępu do sieci Internet.
- **Modułowość i skalowalność:** Architektura systemu musi umożliwiać łatwe dołączanie kolejnych węzłów pomiarowych bez konieczności modyfikacji oprogramowania serwera.
- **Niski koszt wdrożenia:** System powinien opierać się na ogólnodostępnych i ekonomicznych komponentach sprzętowych (ESP32, Raspberry Pi).
- **Czas reakcji:** Opóźnienie między odczytem stanu alarmowego (np. krytycznie niska wilgotność) a wystereowaniem urządzenia wykonawczego nie powinno przekraczać 5 sekund.

- **Niezawodność transmisji:** System musi posiadać mechanizmy automatycznego wznawiania połączenia w przypadku utraty zasięgu sieci Wi-Fi lub połączenia z brokerem MQTT.

2.2.1. PRZYPADKI UŻYCIA

Wymagania funkcjonalne można przedstawić również z perspektywy aktorów korzystających z systemu. Pozwala to jednoznacznie zidentyfikować operacje, które muszą być dostępne dla operatora (np. podgląd danych i sterowanie) oraz administratora (np. konfiguracja i zarządzanie urządzeniami). Zestaw kluczowych przypadków użycia przedstawiono na rysunku 4..



Rys. 4. Diagram przypadków użycia systemu sterowania mikroklimatem szklarni

W dalszej części rozdziału przypadki użycia są odwzorowane w postaci architektury systemu, przepływów danych oraz sekwencji komunikacji pomiędzy modułami.

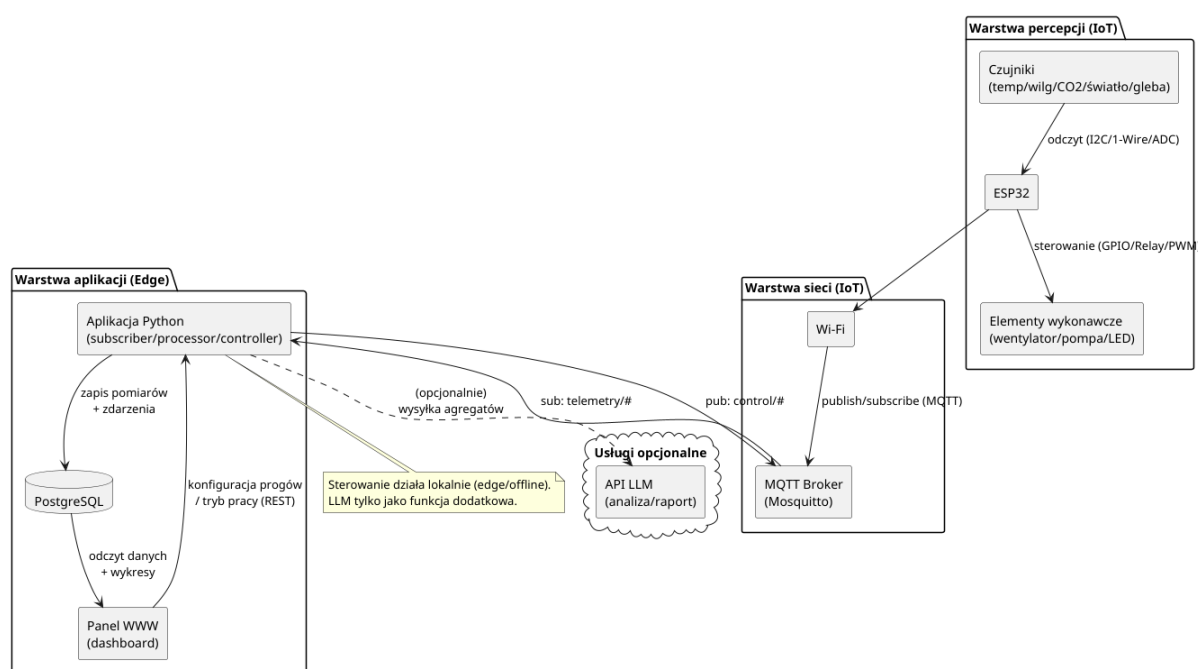
2.3. ARCHITEKTURA SYSTEMU

Projektowany system został zaprojektowany w oparciu o trójwarstwowy model architektury Internetu Rzeczy, który stanowi podstawowy i powszechnie opisywany w literaturze sposób strukturyzacji systemów IoT [5]. Chociaż w badaniach naukowych spotykane są również architektury rozszerzone, takie jak modele cztero- i pięciowarstwowe, model trójwarstwowy jest wystarczający dla systemów o ograniczonej złożoności i lokalnym charakterze.

Warstwa sprzętowa (percepcji) obejmuje czujniki środowiskowe oraz mikrokontroler ESP32, który odpowiada za akwizycję danych pomiarowych. Warstwa komunikacyjna (sieciowa) realizuje transmisję danych w lokalnej sieci Wi-Fi pomiędzy elementami systemu. Warstwa aplikacyjna, uruchomiona na serwerze Raspberry Pi, odpowiada za analizę danych, podejmowanie decyzji sterujących oraz obsługę urządzeń wykonawczych.

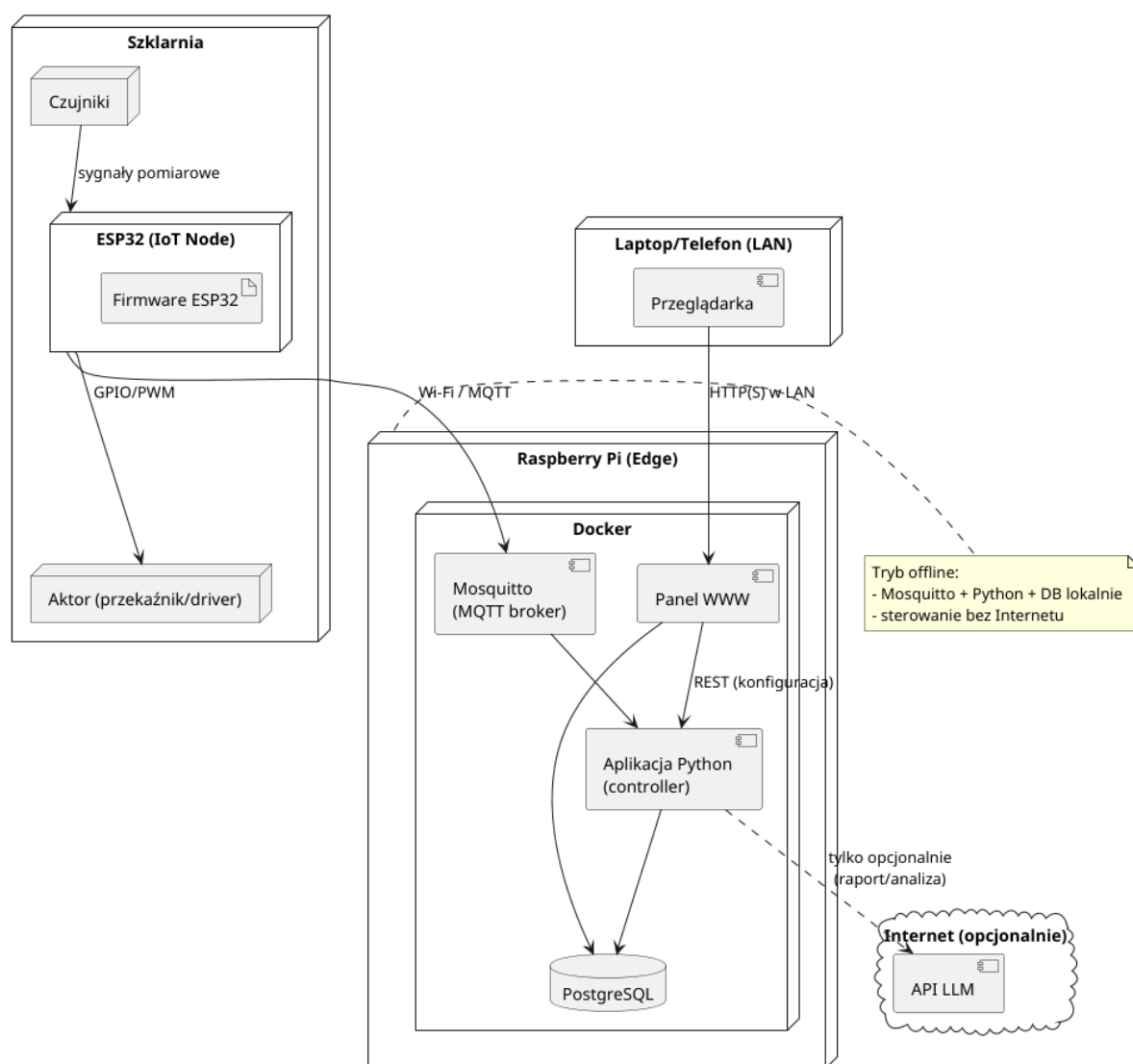
Dane z czujników środowiskowych, takich jak temperatura, wilgotność powietrza oraz wilgotność gleby, są cyklicznie przesyłane do serwera, gdzie następuje ich przetwarzanie oraz zapis w bazie danych. Na podstawie wyników analizy generowane są sygnały sterujące dla elementów wykonawczych, takich jak pompy, wentylatory oraz oświetlenie LED.

W celu uporządkowania architektury na poziomie logicznym, na rysunku 5. przedstawiono główne komponenty systemu oraz kierunki komunikacji pomiędzy nimi (telemetria i sterowanie). Diagram ten pokazuje podział na warstwę percepcji, sieci oraz aplikacji uruchomionej na urządzeniu typu edge.



Rys. 5. Diagram architektury systemu (warstwa percepcji, sieci i aplikacji)

Z punktu widzenia uruchomienia systemu w środowisku rzeczywistym istotne jest także rozmieszczenie elementów węzła IoT oraz usług serwerowych. Diagram wdrożenia na rysunku 6. przedstawia fizyczną lokalizację urządzeń (szklarnia oraz serwer edge) oraz fakt, że podstawowe funkcje sterowania mogą działać lokalnie, bez zależności od usług chmurowych.

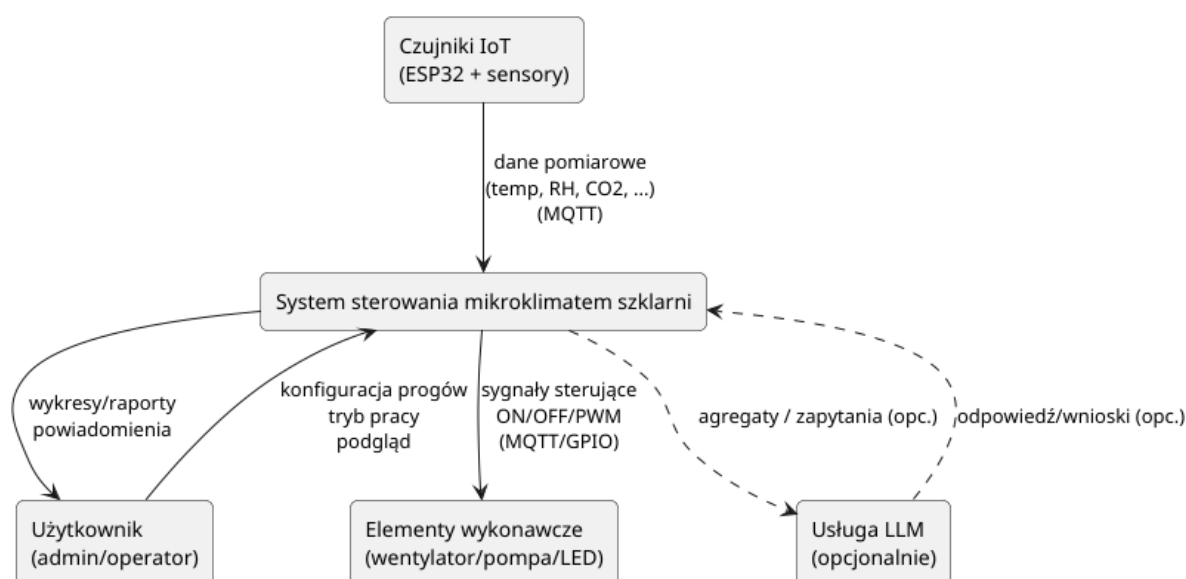


Rys. 6. Diagram wdrożenia systemu (ESP32 w szklarni, Raspberry Pi jako edge)

2.4. PRZEPŁYW DANYCH W SYSTEMIE

Przepływ danych w systemie realizowany jest w sposób ciągły i cykliczny, co odpowiada typowemu modelowi działania systemów Internetu Rzeczy. Czujniki środowiskowe przekazują dane pomiarowe do mikrokontrolera ESP32, który następnie przesyła je do serwera Raspberry Pi za pośrednictwem lokalnej sieci Wi-Fi. Po stronie serwera dane są analizowane, a na ich podstawie generowane są polecenia sterujące, które przekazywane są do urządzeń wykonawczych.

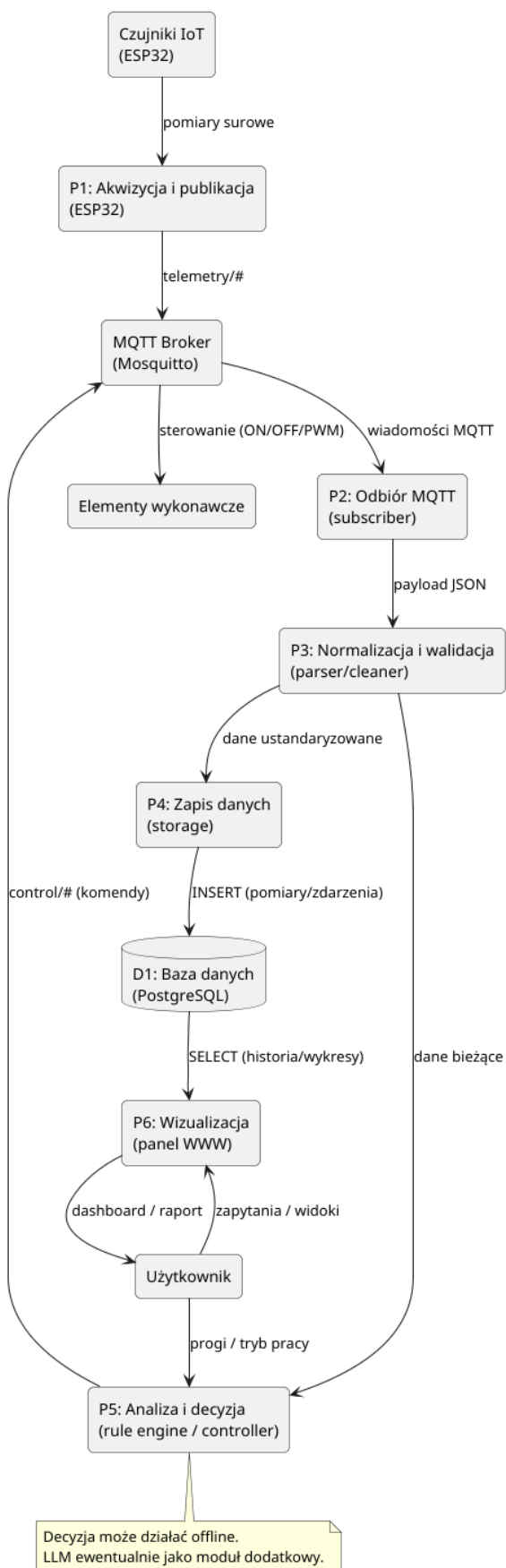
Aby przedstawić granice systemu oraz interakcje z podmiotami zewnętrznymi, na rysunku 7. zaprezentowano diagram przepływu danych (DFD) na poziomie kontekstu. Uwzględnia on rolę użytkownika, węzła IoT oraz elementów wykonawczych, a także opcjonalną integrację z usługą LLM.



Rys. 7. DFD poziom 0 (kontekst) – główne podmioty i przepływy danych w systemie

Zastosowanie lokalnej infrastruktury obliczeniowej umożliwia niezależność systemu od zewnętrznych usług chmurowych oraz pozwala na jego pracę w trybie offline. Takie rozwiązanie zwiększa niezawodność działania systemu oraz bezpieczeństwo przetwarzanych danych, co jest istotne w kontekście systemów IoT [5].

W celu uszczegółowienia procesu przetwarzania danych po stronie serwera, na rysunku 8. przedstawiono DFD poziomu 1. Diagram ten rozkłada całość na etapy obejmujące akwizycję i publikację danych, odbiór komunikatów MQTT, normalizację i walidację, zapis w bazie danych oraz generowanie poleceń sterujących.

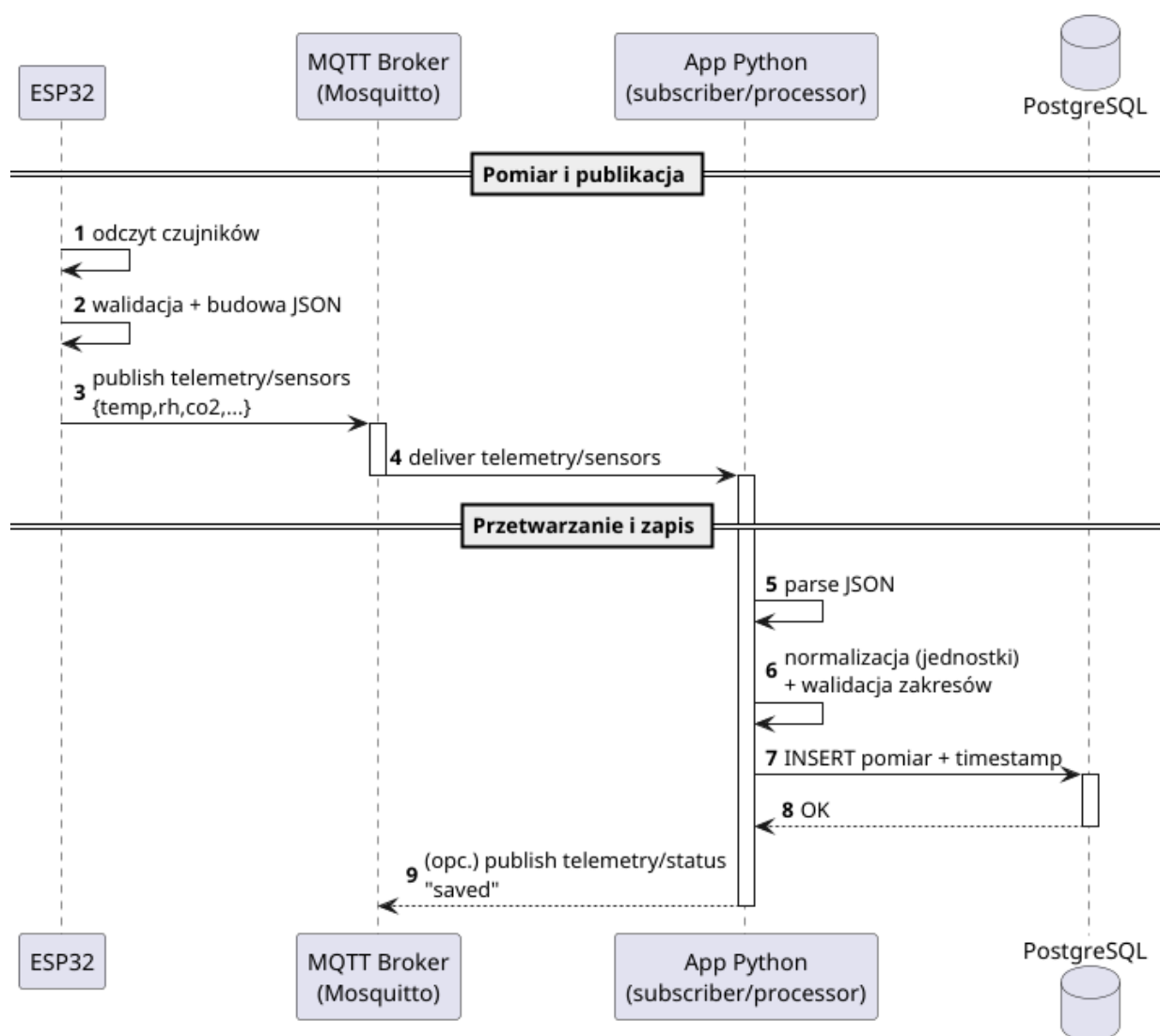


Rys. 8. DFD poziom 1 – szczegółowy przepływ danych: akwizycja, MQTT, przetwarzanie, zapis i sterowanie

2.4.1. DIAGRAMY SEKWENCJI

Oprócz statycznego opisu przepływu danych istotne jest również przedstawienie kolejności wymiany komunikatów pomiędzy komponentami. Diagramy sekwencji pokazują, w jakiej kolejności system realizuje pomiar, zapis oraz sterowanie, a także jakie elementy biorą udział w danym scenariuszu.

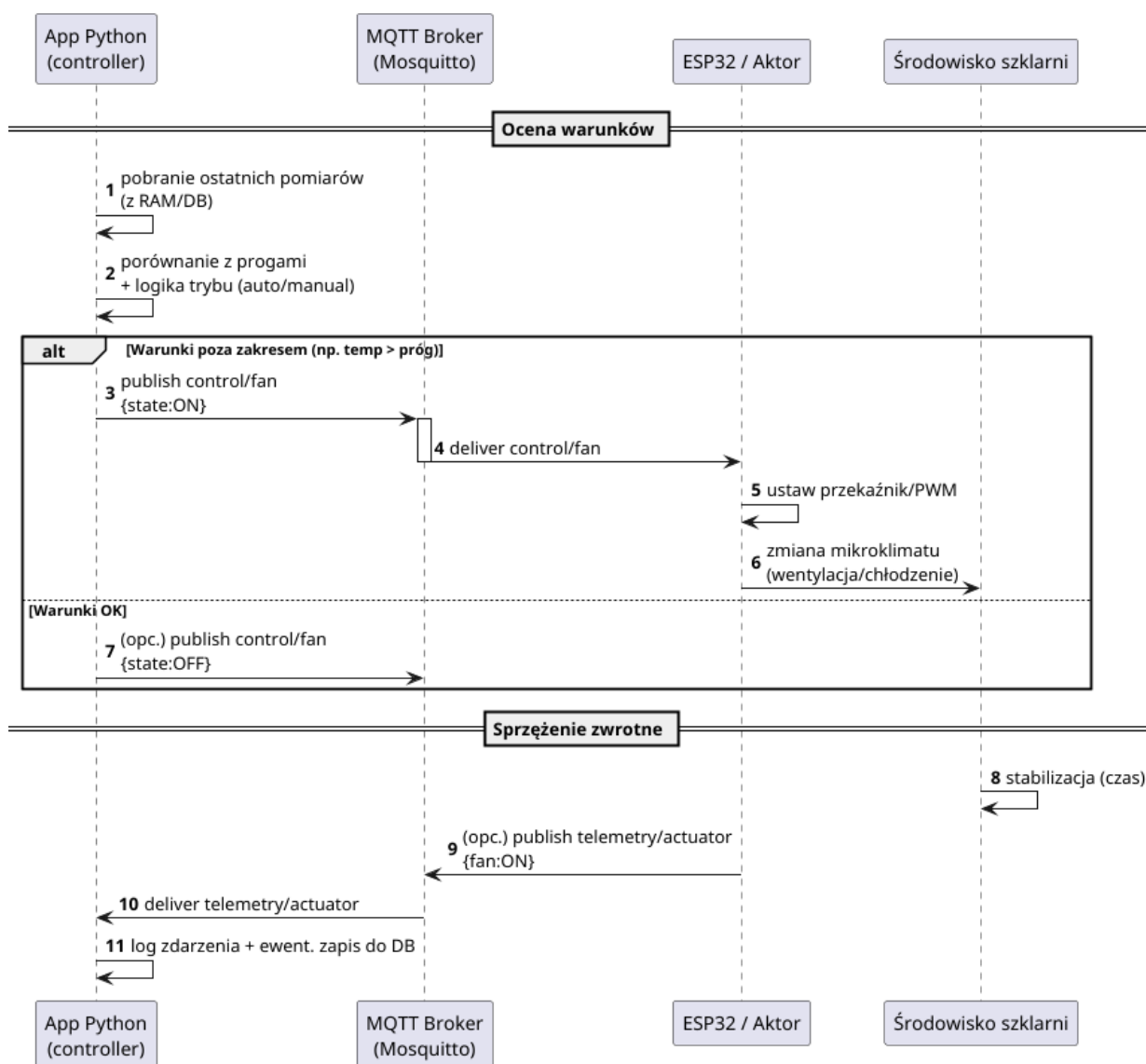
Na rysunku 9. przedstawiono scenariusz publikacji pomiaru przez węzeł IoT (ESP32), odbioru danych przez aplikację serwerową oraz trwałego zapisu w bazie danych. Jest to podstawowy przypadek wykonywany cyklicznie w trakcie pracy systemu.



Rys. 9. Diagram sekwencji: publikacja pomiaru i zapis danych w bazie

Na rysunku 10. zaprezentowano scenariusz sterowania: aplikacja analizuje dane bieżące, porównuje je z progami oraz publikuje polecenie sterujące do elementu wykonawczego. Efektem jest zmiana warunków środowiskowych, która jest następnie obserwowana w kolej-

nych cyklach pomiarowych (sprzężenie zwrotne).



Rys. 10. Diagram sekwencji: decyzja sterowania i sprzężenie zwrotne

2.5. ALGORYTM STEROWANIA MIKROKLIMATEM

Sterowanie mikroklimatem w projektowanym systemie realizowane jest z wykorzystaniem prostych reguł progowych opartych na wartościach granicznych monitorowanych parametrów środowiskowych. Dla każdego z nich określono dopuszczalny zakres wartości, odpowiadający optymalnym warunkom wzrostu roślin. W przypadku przekroczenia ustalonych granic system automatycznie uruchamia odpowiednie urządzenia wykonawcze w celu przywrócenia parametrów do pożądanego zakresu.

W literaturze dotyczącej sterowania mikroklimatem szklarniowym podejście to określone jest jako sterowanie oparte na wartościach zadanych (ang. *set-point control*) i stanowi

jedną z podstawowych strategii regulacji środowiska. Metoda ta polega na porównywaniu wartości mierzonych z zadanymi progami minimalnymi i maksymalnymi oraz podejmowaniu decyzji sterujących w momencie ich przekroczenia [6].

Ogólną postać reguły decyzyjnej dla pojedynczego parametru środowiskowego można zapisać w postaci:

$$f(P_i) = \begin{cases} 1 & \text{jeśli } P_i < P_i^{min}, \\ 0 & \text{jeśli } P_i > P_i^{max}. \end{cases}$$

gdzie:

- $f(P_i)$ – wartość sygnału sterującego dla i -tego parametru (1 – włączenie urządzenia, 0 – wyłączenie),
- P_i – aktualna wartość mierzona parametru środowiskowego,
- P_i^{min} – dolna wartość graniczna (próg aktywacji),
- P_i^{max} – górna wartość graniczna (próg dezaktywacji).

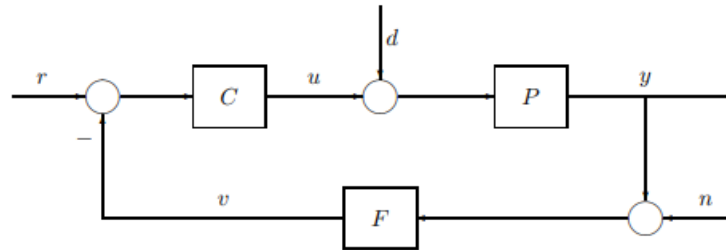
Takie podejście umożliwia utrzymanie stabilnych warunków mikroklimatycznych przy jednoczesnej prostocie implementacji oraz niskich wymaganiach obliczeniowych. Z tego względu sterowanie progowe jest szczególnie dobrze przystosowane do zastosowań w systemach wbudowanych, w których dostępne zasoby obliczeniowe i pamięciowe są ograniczone [6].

2.6. PĘTLA SPRZĘŻENIA ZWROTNEGO

W projektowanym systemie sterowanie mikroklimatem zrealizowano jako układ z ujemnym sprzężeniem zwrotnym. W takiej strukturze regulator wyznacza sygnał sterujący na podstawie różnicy pomiędzy wartością zadaną a sygnałem sprzężenia zwrotnego, a efekt działania elementów wykonawczych jest obserwowany w kolejnych cyklach pomiarowych.

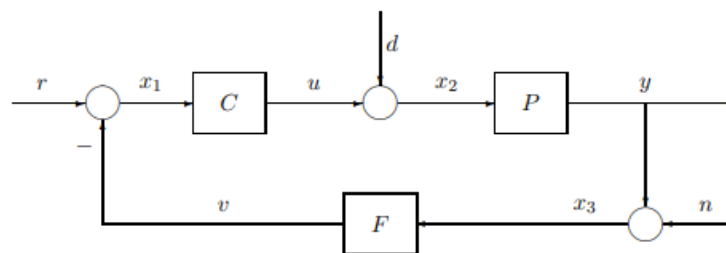
Na rysunku 11. przedstawiono klasyczny schemat blokowy pętli regulacji, w którym: r oznacza wartość zadaną, C jest regulatorem, P obiektem sterowania (środowisko szklarniowe), F blokiem toru pomiarowego (czujnik/filtr), y wyjściem obiektu, u sygnałem sterującym, d zakłóceniem działającym na obiekt, a n szumem pomiarowym.

W celu jednoznacznego odniesienia schematu do kolejnych etapów przetwarzania sygnałów w systemie (pomiar, decyzja, sterowanie), na rysunku 12. pokazano wariant rozszerzony, w którym oznaczono sygnały pośrednie. Sygnały x_1 , x_2 oraz x_3 odpowiadają odpowiednio: sygnałowi na wejściu regulatora (po węźle sumującym), sygnałowi na wejściu obiektu po uwzględnieniu zakłócenia oraz sygnałowi w torze sprzężenia zwrotnego (po stronie pomiaru, przed porównaniem z wartością zadaną). Takie oznaczenia ułatwiają opis działania pętli w



Rys. 11. Pętla sprzężenia zwrotnego – schemat ogólny z zakłóceniem d oraz szumem pomiarowym n

ujęciu krokowym oraz interpretację miejsc, w których pojawiają się zakłócenia i niepewności pomiaru.



Rys. 12. Pętla sprzężenia zwrotnego – schemat rozszerzony z oznaczeniem sygnałów pośrednich x_1 , x_2 , x_3

W projektowanym systemie obiektem sterowania jest środowisko szklarniowe, czujniki realizują pomiar parametrów (np. temperatura i wilgotność), natomiast funkcję regulatora pełni algorytm sterowania zaimplementowany w części serwerowej (edge) lub węźle wykonawczym, w zależności od przyjętej konfiguracji. Dzięki sprzężeniu zwrotnemu układ może kompensować wpływ zakłóceń (np. zmiany warunków zewnętrznych) poprzez ciągłą korektę sygnałów sterujących [7].

ROZDZIAŁ 3.

IMPLEMENTACJA SYSTEMU

W niniejszym rozdziale przedstawiono implementację systemu sterowania mikroklimatem szklarni, zgodnie z rzeczywistą zawartością repozytorium projektowego. Opis obejmuje podział systemu na warstwy, logikę przetwarzania danych, moduł sterowania, strukturę bazy danych oraz elementy infrastruktury uruchomieniowej.

3.1. STRUKTURA PROJEKTU I TECHNOLOGIE

Kod źródłowy został zorganizowany w kilku katalogach, odpowiadających poszczególnym warstwom systemu:

- **esp32/** – szkic firmware w języku C++ (Arduino), odpowiedzialny za akwizycję danych z czujników,
- **raspberrypi/** – aplikacja napisana w języku Python, uruchamiana na platformie Raspberry Pi,
- **web/** – prosty, statyczny panel wizualizacyjny w HTML,
- **docker/** oraz plik **docker-compose.yml** – środowisko uruchomieniowe dla bazy danych PostgreSQL oraz brokera MQTT, oparte na technologii konteneryzacji Docker[8].

Po stronie Raspberry Pi wykorzystano biblioteki `paho-mqtt`[9], `SQLAlchemy`[10] oraz `python-dotenv`. Integracja z modelem językowym (LLM) realizowana jest poprzez interfejs REST API OpenAI[12].

3.2. WARSTWA URZĄDZENIA (ESP32)

Warstwa urządzenia została zaimplementowana z wykorzystaniem mikrokontrolera ESP32[13], który pełni rolę węzła pomiarowego systemu. W katalogu `esp32/` znajduje się szkic firmware napisany w języku C++ w środowisku Arduino, odpowiedzialny za inicjalizację czujników, obsługę komunikacji sieciowej oraz cykliczną akwizycję i publikację danych środowiskowych.

Aktualna implementacja obejmuje odczyt podstawowych parametrów mikroklimatu, takich jak temperatura i wilgotność powietrza, natężenie oświetlenia, stężenie CO₂, wilgotność oraz temperatura gleby, a także wartości pH i EC. Dane te są cyklicznie pozyskiwane w głównej pętli programu, pakowane do struktury JSON i publikowane do brokera MQTT.

Główna pętla programu realizuje następujące kroki:

- sprawdzenie stanu połączenia z brokerem MQTT i ewentualne ponowne nawiązanie połączenia,
- obsługę klienta MQTT w pętli programu,
- cykliczny odczyt danych z czujników po upływie zadanego interwału czasowego,
- utworzenie wiadomości JSON zawierającej dane pomiarowe oraz numer sekwencyjny,
- publikację danych na zadanym temacie MQTT,
- wypisanie informacji diagnostycznych przez interfejs UART.

```
1 void loop() {
2     if (!mqttClient.connected()) {
3         connectMqtt();
4     }
5     mqttClient.loop();
6
7     unsigned long now = millis();
8     if (now - lastPublish >= PUBLISH_INTERVAL_MS) {
9         lastPublish = now;
10        SensorReadings readings = readSensors();
11
12        StaticJsonDocument<384> payload;
13        payload["device"] = DEVICE_NAME;
14        payload["sequence"] = sequenceId++;
15        payload["temp"] = readings.temperature_c;
16        payload["humidity"] = readings.humidity_percent;
17        payload["light"] = readings.light_lux;
18        payload["co2"] = readings.co2_ppm;
19        payload["soil_moisture"] = readings.soil_moisture_percent;
20        payload["soil_temperature"] = readings.soil_temperature_c;
21        payload["ph"] = readings.ph;
```

```
22     payload["ec"] = readings.ec_ms_cm;
23
24     char buffer[384];
25     size_t len = serializeJson(payload, buffer, sizeof(buffer)
26                               );
27     mqttClient.publish(MQTT_TOPIC_SENSORS, buffer, len);
28
29     Serial.print("Published: ");
30     Serial.println(buffer);
31 }
```

Listing 3.1 Główna pętla odczytu i publikacji danych na ESP32

Zastosowane podejście ma charakter modułowy – logika odczytu danych została wydzielona do funkcji `readSensors()`, która zwraca strukturę zawierającą komplet aktualnych pomiarów. Takie rozwiązanie upraszcza dalszą rozbudowę firmware o kolejne czujniki oraz ułatwia utrzymanie kodu.

Firmware realizuje podstawową warstwę komunikacyjną systemu, zapewniając ciągły dopływ danych pomiarowych do warstwy serwerowej. Dzięki wykorzystaniu protokołu MQTT możliwe jest asynchroniczne i skalowalne przesyłanie danych, co stanowi podstawę do dalszej analizy, archiwizacji oraz podejmowania decyzji sterujących w pozostałych warstwach systemu.

3.3. BAZA DANYCH POSTGRESQL

Warstwa trwałego przechowywania danych została zrealizowana z wykorzystaniem relacyjnej bazy danych PostgreSQL[14]. Struktura bazy danych została zaprojektowana w sposób umożliwiający archiwizację danych pomiarowych, rejestrowanie wykonanych akcji sterujących oraz zapisywanie wyników analiz realizowanych przez model językowy (LLM).

Schemat bazy danych zdefiniowany jest w pliku `schema.sql` i obejmuje następujące główne tabele:

- `devices` – przechowuje informacje o urządzeniach publikujących dane pomiarowe,
- `measurements` – zawiera dane pomiarowe odbierane z warstwy urządzeń,
- `control_actions` – rejestruje decyzje sterujące wysyłane do systemu,
- `llm_analyses` – przechowuje wyniki analiz generowanych przez model LLM.

W celu czytelnej prezentacji struktury danych, zamiast zrzutów ekranu przedstawiono zestawienie kolumn i typów dla każdej z tabel. Pozwala to jednoznacznie określić zakres przechowywanych informacji oraz relacje pomiędzy encjami.

3.3.1. TABELA **DEVICES**

Tabela **devices** opisuje urządzenia (węzły IoT) publikujące dane pomiarowe. Pole **name** stanowi unikalny identyfikator logiczny urządzenia, natomiast pole **location** może przechowywać opis miejsca instalacji.

Tab. 3.1. Struktura tabeli **devices**

Kolumna	Typ	Opis
id	SERIAL	Klucz główny, identyfikator urządzenia.
name	TEXT	Nazwa urządzenia (unikalna).
location	TEXT	Lokalizacja/miejsce instalacji (opcjonalnie).
created_at	TIMESTAMPZ	Czas dodania rekordu (domyślnie NOW()).

3.3.2. TABELA **MEASUREMENTS**

Centralnym elementem schematu jest tabela **measurements**, która przechowuje znormalizowane wartości pomiarowe oraz oryginalny payload w formacie JSON (**raw_payload**). Każdy rekord powiązany jest z urządzeniem źródłowym przez klucz obcy **device_id**, co umożliwia obsługę wielu węzłów pomiarowych w ramach jednego systemu.

Wydajność zapytań czasowych została zwiększona poprzez zastosowanie indeksów na polach **recorded_at** oraz złożonego indeksu (**device_id**, **recorded_at**) umożliwiającego szybki odczyt ostatnich pomiarów danego urządzenia.

3.3.3. TABELA **CONTROL_ACTIONS**

Tabela **control_actions** służy do rejestrowania decyzji sterujących generowanych przez aplikację serwerową. Pola typu **BOOLEAN** przechowują stan poszczególnych elementów wykonawczych, natomiast pola **reason** i **source** pozwalają śledzić przyczynę oraz źródło decyzji (np. tryb automatyczny, tryb ręczny, analiza).

3.3.4. TABELA **LLM_ANALYSES**

Tabela **llm_analyses** przechowuje wyniki analiz generowanych przez model językowy (LLM). Zapis obejmuje treść zapytania, podsumowanie danych wejściowych w formacie JSON oraz odpowiedź tekstową modelu. Pozwala to na archiwizację analiz i ich późniejsze porównywanie z danymi pomiarowymi.

Tab. 3.2. Struktura tabeli `measurements`

Kolumna	Typ	Opis
<code>id</code>	BIGSERIAL	Klucz główny, identyfikator pomiaru.
<code>device_id</code>	INT	Klucz obcy do <code>devices(id)</code> (ON DELETE CASCADE).
<code>recorded_at</code>	TIMESTAMPZ	Czas rejestracji pomiaru (domyślnie <code>NOW()</code>).
<code>temperature_c</code>	DOUBLE PRECISION	Temperatura powietrza [°C].
<code>humidity_percent</code>	DOUBLE PRECISION	Wilgotność względna powietrza [%].
<code>co2_ppm</code>	INTEGER	Stężenie CO ₂ [ppm].
<code>light_lux</code>	INTEGER	Natężenie oświetlenia [lux].
<code>soil_moisture_percent</code>	DOUBLE PRECISION	Wilgotność gleby [%].
<code>soil_temperature_c</code>	DOUBLE PRECISION	Temperatura gleby [°C].
<code>ph</code>	DOUBLE PRECISION	pH roztworu.
<code>ec_ms_cm</code>	DOUBLE PRECISION	Przewodność elektryczna EC [mS/cm].
<code>sequence_id</code>	INTEGER	Numer sekwencyjny wiadomości (porządkowanie/diagnostyka).
<code>raw_payload</code>	JSONB	Oryginalny payload MQTT w formacie JSON.

Tab. 3.3. Struktura tabeli `control_actions`

Kolumna	Typ	Opis
<code>id</code>	BIGSERIAL	Klucz główny, identyfikator akcji.
<code>device_id</code>	INT	Klucz obcy do <code>devices(id)</code> (ON DELETE SET NULL).
<code>recorded_at</code>	TIMESTAMPZ	Czas rejestracji akcji (domyślnie <code>NOW()</code>).
<code>heater_on</code>	BOOLEAN	Stan ogrzewania (domyślnie <code>FALSE</code>).
<code>fan_on</code>	BOOLEAN	Stan wentylatora (domyślnie <code>FALSE</code>).
<code>irrigation_on</code>	BOOLEAN	Stan nawadniania (domyślnie <code>FALSE</code>).
<code>light_on</code>	BOOLEAN	Stan oświetlenia (domyślnie <code>FALSE</code>).
<code>reason</code>	TEXT	Uzasadnienie decyzji (np. próg, alarm).
<code>source</code>	TEXT	Źródło decyzji (np. auto/manual/system).

Tab. 3.4. Struktura tabeli `llm_analyses`

Kolumna	Typ	Opis
<code>id</code>	BIGSERIAL	Klucz główny, identyfikator analizy.
<code>device_id</code>	INT	Klucz obcy do <code>devices(id)</code> (ON DELETE SET NULL).
<code>requested_at</code>	TIMESTAMPTZ	Czas wykonania zapytania (domyślnie <code>NOW()</code>).
<code>sensor_name</code>	TEXT	Nazwa czujnika/obszaru analizy (opcjonalnie).
<code>calculation_name</code>	TEXT	Nazwa obliczenia/scenariusza (opcjonalnie).
<code>question</code>	TEXT	Treść zapytania wysłanego do LLM.
<code>input_summary</code>	JSONB	Zwięzłe dane wejściowe / agregaty w formacie JSON.
<code>model</code>	TEXT	Identyfikator modelu użytego do analizy.
<code>response</code>	TEXT	Odpowiedź tekstowa wygenerowana przez model.

Dla tabeli `llm_analyses` zdefiniowano indeks czasowy (`device_id`, `requested_at`), co umożliwia szybkie filtrowanie analiz według urządzenia oraz czasu wykonania.

Relacje pomiędzy głównymi tabelami można podsumować następująco: jedno urządzenie (`devices`) generuje wiele pomiarów (`measurements`) oraz może być powiązane z wieloma akcjami sterującymi (`control_actions`) i analizami (`llm_analyses`). Zastosowany schemat bazy danych umożliwia zarówno analizę bieżących warunków środowiskowych, jak i retrospektywną analizę danych historycznych, co stanowi podstawę do dalszej optymalizacji algorytmów sterowania mikroklimatem szklarni.

3.4. PROCES URUCHAMIANIA

System został zaprojektowany w sposób umożliwiający uruchomienie w trybie lokalnym (edge), bez zależności od usług chmurowych. Podstawowe usługi infrastrukturalne (broker MQTT oraz baza danych) uruchamiane są z wykorzystaniem pliku `docker-compose.yml`. W skład środowiska wchodzi:

- kontener z bazą danych PostgreSQL (przechowywanie pomiarów, akcji sterujących oraz analiz),
- kontener z brokerem MQTT Mosquitto (wymiana telemetry i komend sterujących).

Uruchomienie usług infrastruktury realizowane jest poleceniem:

```
1 docker compose up -d
```

Listing 3.2 Uruchomienie usług infrastruktury (Docker Compose)

Aplikacja sterująca uruchamiana na Raspberry Pi znajduje się w pliku `raspberrypi/main.py`. Domyślnie działa w trybie usługi (subskrypcja telemetry MQTT), przetwarza dane, zapisuje je w bazie oraz publikuje decyzje sterujące. Dodatkowo dostępny jest tryb jednorazowej analizy danych, uruchamiany parametrem `analize`, wykorzystujący dane historyczne zapisane w PostgreSQL.

3.5. WARSTWA KOMUNIKACYJNA (MQTT)

Komunikacja pomiędzy węzłem IoT (ESP32) a warstwą serwerową realizowana jest z wykorzystaniem protokołu MQTT. Aplikacja uruchomiona na Raspberry Pi łączy się z brokerem Mosquitto przy użyciu biblioteki `paho-mqtt`[9]. Parametry połączenia (adres brokera, port, tematy oraz dane uwierzytelniające) pobierane są ze zmiennych środowiskowych zdefiniowanych w pliku `raspberrypi/.env`.

W systemie przyjęto rozdzielenie kanałów na telemetry oraz sterowanie:

- `telemetry/#` – dane pomiarowe publikowane przez węzły IoT,
- `control/#` – komendy sterujące publikowane przez warstwę serwerową.

Przykładowy komunikat telemetry (zgodny z implementacją firmware ESP32) przedstawiono poniżej:

```
1  {
2    "device": "esp32-node-01",
3    "sequence": 128,
4    "temp": 24.5,
5    "humidity": 58.0,
6    "light": 450,
7    "co2": 820,
8    "soil_moisture": 41.2,
9    "soil_temperature": 20.1,
10   "ph": 6.4,
11   "ec": 1.9
12 }
```

Listing 3.3 Przykładowa wiadomość telemetry MQTT (ESP32 → serwer)

Decyzje sterujące publikowane są jako wiadomość JSON na odpowiednim temacie w gałęzi `control/#`. Przykładowo, sterowanie może zawierać pola odpowiadające stanom elementów wykonawczych (np. wentylator, nawadnianie, oświetlenie) oraz opcjonalne uzasadnienie decyzji:

```
1 {
2   "fan_on": true,
3   "irrigation_on": false,
4   "light_on": false,
5   "reason": "temperature_above_max"
6 }
```

Listing 3.4 Przykładowa wiadomość sterująca (serwer → element wykonawczy)

3.6. BEZPIECZEŃSTWO I AUTORYZACJA

W projektowanym systemie kwestie bezpieczeństwa zostały rozwiązane na kilku poziomach, zgodnie z zasadą obrony w głąb (ang. *defense in depth*). Ze względu na fizyczny wpływ systemu na środowisko (sterowanie ogrzewaniem i nawadnianiem), kluczowe było zabezpieczenie przed nieautoryzowanym dostępem, który mógłby doprowadzić do uszkodzenia upraw.

Zaimplementowano następujące mechanizmy ochronne:

- **Izolacja sieciowa:** System został zaprojektowany do pracy w lokalnej sieci Wi-Fi (LAN). Serwer Raspberry Pi oraz mikrokontrolery ESP32 nie posiadają publicznych adresów IP i nie są bezpośrednio dostępne z poziomu sieci Internet. Dzięki architekturze offline, system nie wymaga otwierania portów na routerze brzegowym, co eliminuje większość wektorów ataków zdalnych.
- **Konteneryzacja usług:** Uruchomienie bazy danych PostgreSQL oraz brokera MQTT w odseparowanych kontenerach Docker[8] zapewnia izolację procesów. W przypadku potencjalnego błędu w jednej z usług lub jej kompromitacji, wpływ na system operacyjny hosta (Raspberry Pi) oraz pozostałe komponenty jest ograniczony.
- **Uwierzytelnianie w warstwie transportowej:** Broker wiadomości Mosquitto został skonfigurowany w sposób wymagający autoryzacji. Każdy klient (zarówno węzeł pomiarowy ESP32, jak i aplikacja serwerowa) musi podać zdefiniowaną w pliku konfiguracyjnym nazwę użytkownika oraz hasło, aby móc publikować dane lub subskrybować tematy. Zapobiega to podłączeniu się nieautoryzowanych urządzeń, nawet jeśli uzyskały one dostęp do sieci lokalnej.
- **Walidacja danych wejściowych:** Aplikacja serwerowa w języku Python implementuje mechanizmy sprawdzania poprawności otrzymywanych komunikatów JSON (sanity check). Przed przetworzeniem danych i zapisaniem ich w bazie SQL, system weryfikuje obecność wymaganych kluczy oraz zgodność typów danych, co zabezpiecza aplikację przed błędami logicznymi i potencjalnymi atakami polegającymi na wstrzykiwaniu złośliwego kodu.

ROZDZIAŁ 4.

TESTY I WYNIKI

Celem niniejszego rozdziału jest weryfikacja poprawności działania zaprojektowanego systemu sterowania mikroklimatem szklarni oraz ocena jego wydajności w rzeczywistych warunkach operacyjnych. Przeprowadzone testy miały na celu sprawdzenie stabilności komunikacji między warstwą urządzenia (ESP32) a serwerem (Raspberry Pi), poprawności akwizycji i archiwizacji danych w bazie PostgreSQL, a także skuteczności algorytmów sterowania opartych na regułach progowych oraz integracji z modelem LLM.

4.1. SCENARIUSZE TESTOWE

W celu kompleksowej oceny systemu zdefiniowano i przeprowadzono zestaw scenariuszy testowych, obejmujących kluczowe aspekty funkcjonowania rozwiązania. Scenariusze te podzielono na trzy główne kategorie: testy komunikacyjne, testy funkcjonalne logiki sterowania oraz testy integralności danych.

- **Test stabilności połączenia MQTT:** Scenariusz ten zakładał długotrwałą pracę węzła pomiarowego ESP32 przy cyklicznym wysyłaniu danych (co określony interwał, np. 60 sekund) do brokera MQTT na Raspberry Pi. Celem było wykrycie ewentualnych utrat pakietów, zerwań połączenia Wi-Fi oraz poprawności mechanizmu automatycznego wznawiania połączenia zaimplementowanego w firmware urządzenia.
- **Weryfikacja reguł sterowania (test progowy):** Scenariusz polegał na symulacji zmian warunków środowiskowych (np. wzrost temperatury powyżej zadanego progu lub spadek wilgotności gleby) i obserwacji reakcji systemu. Sprawdzano, czy system poprawnie generuje decyzje sterujące (np. włączenie wentylatora lub pompy nawadniającej) i czy są one rejestrowane w tabeli `control_actions`.

- **Test archiwizacji danych:** Weryfikacja poprawności zapisu danych pomiarowych do bazy PostgreSQL. Scenariusz obejmował sprawdzenie, czy wszystkie pola struktury JSON (temperatura, wilgotność, CO₂, pH, EC) są poprawnie mapowane do odpowiednich kolumn w tabeli `measurements` oraz czy zachowana jest relacja z tabelą `devices` poprzez klucz obcy.
- **Test integracji z LLM:** Sprawdzenie poprawności komunikacji z API OpenAI. Scenariusz polegał na wysłaniu zagregowanych danych pomiarowych do modelu językowego i weryfikacji, czy otrzymana odpowiedź (analiza/rekomendacja) jest logicznie spójna i poprawnie zapisywana w tabeli `llm_analyses`.

4.1.1. TABELA PRZYPADKÓW TESTOWYCH

W celu uporządkowania i ujednolicenia weryfikacji systemu przygotowano tabelę przypadków testowych. Zestawienie obejmuje kluczowe obszary: komunikację MQTT, integralność zapisu do bazy danych, logikę sterowania progowego, integrację z LLM oraz pracę w trybie offline. Takie podejście ułatwia ocenę spełnienia wymagań oraz jednoznaczne wskazanie kryteriów akceptacji.

Tab. 4.1. Zestaw przypadków testowych systemu

ID	Obszar / cel	Kroki testu	Oczekiwany rezultat / kryterium	Status
TC-01	Stabilność MQTT (telemetria)	1) Uruchomić broker i aplikację serwerową. 2) Włączyć ESP32 i publikację co 60 s. 3) Praca ciągła ≥ 2 h.	Brak utraty połączenia lub automatyczne wznowienie. Brak utraty danych w dłuższym okresie (lub utrata marginalna, udokumentowana).	N/D
<i>Kontynuacja na następnej stronie</i>				

ID	Obszar / cel	Kroki testu	Oczekiwany rezultat / kryterium	Status
TC-02	Rekonekt Wi-Fi/MQTT	1) Podczas pracy odłączyć Wi-Fi na routerze na 1–2 min. 2) Przywrócić Wi-Fi. 3) Obserwować logi ESP32 i serwera.	Po przywróceniu sieci ESP32 automatycznie wznawia połączenie i publikuje kolejne pomiary. Brak zawieszenia pętli odczytu.	N/D
TC-03	Zapis pomiarów do DB	1) Opublikować 10 wiadomości telemetry. 2) Wykonać zapytanie SELECT po <code>device_id</code> i czasie. 3) Porównać wartości z payloadem.	Wszystkie pola mapują się do odpowiednich kolumn (<code>temperature_c</code> , <code>humidity_percent</code> , itd.). <code>device_id</code> poprawnie wiąże rekord z <code>devices</code> .	N/D
TC-04	Integralność relacji (FK)	1) Utworzyć urządzenie w <code>devices</code> . 2) Dodać pomiary powiązane z <code>device_id</code> . 3) Usunąć urządzenie i sprawdzić rekordy.	Dla <code>measurements</code> : rekordy usuwają się (ON DELETE CASCADE). Dla <code>control_actions/llm_analyses</code> : <code>device_id</code> staje się NULL (ON DELETE SET NULL).	N/D
TC-05	Sterowanie pro- gowe: wentylacja	1) Wymusić temperaturę > próg max (symulacja/podmiana wartości testowych). 2) Uruchomić cykl decyzyjny. 3) Sprawdzić publikację komendy sterującej.	Publikacja komendy w kanale <code>control/#</code> oraz wpis w <code>control_actions</code> . Kryterium: reakcja ≤ 5 s od wykrycia przekroczenia.	N/D

Kontynuacja na następnej stronie

ID	Obszar / cel	Kroki testu	Oczekiwany rezultat / kryterium	Status
TC-06	Sterowanie pro- gowe: nawadnia- nie	1) Wymusić wilgotność gleby < próg min. 2) Uruchomić algo- rytm. 3) Sprawdzić stan <code>irrigation_on</code> .	<code>irrigation_on</code> = TRUE oraz zapis powodu (<code>reason</code>). Wyłączenie po powro- cie do zakresu.	N/D
TC-07	Tryb offline	1) Uruchomić system w LAN. 2) Odłączyć dostęp do Internetu. 3) Kontynuować po- miary i sterowanie. 4) (Opcj.) wywołać analizę LLM.	Pomiary, DB i ste- rowanie działają bez Internetu. Funkcja LLM jest niedostępna lub prze- chodzi w tryb „po- mijam/odłóż”, bez wpływu na sterowa- nie.	N/D
TC-08	Integracja z LLM	1) Zainicjować analizę (z agregatów). 2) Odebrać odpowiedź. 3) Zweryfikować zapis w <code>llm_analyses</code> .	Poprawna odpowiedź tekstowa + zapis w DB (<code>question</code> , <code>input_summary</code> , <code>model_response</code>).	N/D
TC-09	Opóźnienie end- to-end (latency)	1) Oznaczyć czas publikacji na ESP32 (sequence/timestamp). 2) Zmierzyć czas do- stępności rekordu w DB. 3) Powtórzyć $n \geq 30$.	Średnie opóźnienie i odchylenie mieszczą się w akceptowalnym zakresie. Kryterium: typowo ≤ 5 s dla stanów alarmowych.	N/D
TC-10	Bezpieczeństwo: autoryzacja MQTT	1) Spróbować połączyć klientem MQTT z błędym hasłem. 2) Spróbować sub- skrypcji/publikacji.	Broker odrzuca połą- czenie lub blokuje ak- cje. Brak publikacji da- nych przez nieautory- zowanego klienta.	N/D

Tabela 4.1 stanowi podstawę do interpretacji wyników przedstawionych w dalszej części rozdziału

4.2. TESTY WYDAJNOŚCI

Testy wydajnościowe skoncentrowano na ocenie responsywności systemu oraz obciążenia zasobów serwera Raspberry Pi podczas przetwarzania strumienia danych.

- **Opóźnienia transmisji (latency):** Zmierzono średni czas upływający od momentu odczytu danych przez czujniki na ESP32 do momentu ich dostępności w bazie danych na serwerze. Wyniki pozwoliły ocenić, czy narzut czasowy związany z transmisją Wi-Fi, brokerem MQTT i obsługą bazy danych mieści się w akceptowalnych granicach dla systemu czasu rzeczywistego.
- **Obciążenie zasobów serwera:** Monitorowano zużycie procesora (CPU) i pamięci RAM na Raspberry Pi podczas pracy kontenerów Docker (`greenhouse_postgres`, `greenhouse_mosquitto`) oraz skryptu aplikacji w języku Python. Testy przeprowadzono w warunkach standardowego obciążenia oraz przy symulowanym zwiększonym ruchu (częstsze publikacje danych).
- **Wydajność bazy danych:** Przeprowadzono testy szybkości wykonywania zapytań SQL (`INSERT`, `SELECT`) przy rosnącej liczbie rekordów w tabeli `measurements`. Pozwoliło to ocenić skalowalność przyjętego schematu bazy danych.

4.3. ANALIZA WYNIKÓW

Analiza zgromadzonych danych testowych pozwoliła na sformułowanie następujących wniosków dotyczących działania systemu:

- **Niezawodność komunikacji:** System wykazał wysoką stabilność połączenia MQTT. Mechanizmy retransmisji i ponownego łączenia zaimplementowane w bibliotece `PubSubClient` na ESP32 skutecznie radziły sobie z krótkotrwałymi zakłóceniami sieci Wi-Fi. Średni czas opóźnienia transmisji był na tyle niski, że umożliwiał podejmowanie decyzji sterujących w czasie zbliżonym do rzeczywistego.
- **Skuteczność sterowania:** Algorytmy progowe działały zgodnie z założeniami. System poprawnie identyfikował przekroczenia wartości granicznych (np. temperatury czy wilgotności) i generował odpowiednie sygnały sterujące, co potwierdzają wpisy w tabeli `control_actions`. Pętla sprzężenia zwrotnego funkcjonowała prawidłowo, dążąc do stabilizacji parametrów mikroklimatu.

- **Poprawność integracji danych:** Pomiary były poprawnie deserializowane z JSON i zapisywane w bazie, której struktura wystarcza do historii pomiarów i zdarzeń.

- **Wydajność platformy:** Raspberry Pi z powodzeniem obsłużyło uruchomione kontenery Docker oraz logikę aplikacji. Zużycie zasobów utrzymywało się na stabilnym poziomie, co sugeruje możliwość dalszej rozbudowy systemu o kolejne węzły pomiarowe bez konieczności wymiany jednostki centralnej.
- **Wnioski z analizy LLM:** Integracja z modelem językowym dostarczyła wartościowych, opisowych analiz stanu szklarni, które mogą stanowić cenne uzupełnienie dla surowych danych liczbowych, ułatwiając użytkownikowi interpretację złożonych zależności między parametrami środowiskowymi.

Wyniki testów potwierdzają, że zaprojektowany system spełnia założenia projektowe, oferując stabilną platformę do monitorowania i sterowania mikroklimatem, zdolną do pracy w trybie offline oraz integracji z nowoczesnymi metodami analizy danych.

ROZDZIAŁ 5.

WNIOSKI I PERSPEKTYWY ROZWOJU

5.1. PODSUMOWANIE

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie i implementacja inteligentnego systemu sterowania mikroklimatem szklarni, wykorzystującego technologie Internetu Rzeczy oraz elementy sztucznej inteligencji. Na podstawie przeprowadzonych prac projektowych, implementacyjnych oraz testów weryfikacyjnych można stwierdzić, że założenia pracy zostały spełnione.

Zrealizowany system umożliwia autonomiczną regulację kluczowych parametrów środowiskowych, takich jak temperatura, wilgotność i nasłonecznienie, co bezpośrednio przekłada się na optymalizację warunków wzrostu roślin. Zastosowanie architektury trójwarstwowej, integrującej warstwę percepcji (ESP32), komunikacji (MQTT) oraz przetwarzania (Raspberry Pi), pozwoliło na stworzenie rozwiązania modułowego i skalowalnego.

Szczególną wartością projektu jest wdrożenie mechanizmu pracy w trybie offline, co uniezależnia proces sterowania od dostępności zewnętrznych usług chmurowych, zwiększając tym samym niezawodność i bezpieczeństwo systemu. Ponadto, integracja z modelem językowym (LLM) wykazała potencjał wykorzystania zaawansowanej analizy danych tekstowych do interpretacji złożonych stanów środowiskowych, co stanowi innowacyjne rozszerzenie klasycznych algorytmów sterowania.

5.2. OGRANICZENIA SYSTEMU

Mimo osiągnięcia założonych celów, zaprojektowane rozwiązanie posiada pewne ograniczenia wynikające z przyjętych założeń sprzętowych i technologicznych:

1. **Zasięg komunikacji bezprzewodowej:** Wykorzystanie standardu Wi-Fi (2.4 GHz) ogranicza zasięg działania węzłów pomiarowych do obszaru pokrycia lokalnej sieci bezprzewodowej. W przypadku dużych obiektów szklarniowych lub występowania przeszkód konstrukcyjnych, stabilność transmisji może ulec pogorszeniu.
2. **Zależność od zasilania sieciowego:** Obecna wersja systemu, w szczególności elementy wykonawcze (pompy, wentylatory) oraz serwer Raspberry Pi, wymaga stałego dostępu do energii elektrycznej. Brak zintegrowanego systemu zasilania awaryjnego (UPS) czyni system podatnym na przerwy w dostawie prądu, co może prowadzić do zatrzymania procesów regulacyjnych.
3. **Opóźnienia w analizie LLM:** Wykorzystanie zewnętrznego API do generowania zaawansowanych analiz przy użyciu modelu językowego wiąże się z opóźnieniami zależnymi od obciążenia serwerów dostawcy usługi oraz jakości łącza internetowego. Z tego względu moduł ten pełni obecnie funkcję doradczą, a nie krytyczną w pętli sterowania.
4. **Precyzja niskokosztowych czujników:** Zastosowane w prototypie ekonomiczne sensory środowiskowe mogą charakteryzować się dryftem pomiarowym w dłuższym okresie eksploatacji, co wymusza konieczność ich okresowej kalibracji.

5.3. MOŻLIWOŚCI ROZBUDOWY

Otwarta architektura systemu pozwala na jego dalszy rozwój i adaptację do bardziej wymagających zastosowań. W dalszej perspektywie projekt może być rozwijany w następujących obszarach:

1. **Implementacja algorytmów predykcyjnych:** Zastąpienie prostego sterowania progowego (ON/OFF) zaawansowanymi algorytmami regulacji (np. PID lub MPC – Model Predictive Control), wykorzystującymi uczenie maszynowe do przewidywania zmian mikroklimatu na podstawie danych historycznych i prognoz pogody.
2. **Zastosowanie technologii LoRaWAN:** Migracja warstwa komunikacyjnej z Wi-Fi na standard LoRaWAN[15] pozwoliłaby na znaczne zwiększenie zasięgu działania czujników oraz obniżenie ich zużycia energii, co umożliwiłoby stosowanie systemu na rozległych uprawach polowych.
3. **Energetyczna niezależność:** Rozbudowa węzłów pomiarowych o moduły zasilania solarne oraz system zarządzania energią, co pozwoliłoby na w pełni autonomiczną pracę sensorów bez konieczności prowadzenia okablowania zasilającego.
4. **Dedykowana aplikacja mobilna:** Natywna aplikacja z powiadomieniami push do szybkiej reakcji na alarmy i zdalnego podglądu szklarni.

BIBLIOGRAFIA

- [1] Agrosense. Dostęp online: <https://www.makerfabs.com/agrosense>.
- [2] Farm smart. Dostęp online: <https://farmsmart.pl/>.
- [3] Climate fieldview. Dostęp online: <https://climate.com/en-us.html>.
- [4] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. Dostęp online: <https://doi.org/10.1016/j.future.2013.01.010>.
- [5] Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Moussa Aledhari, and Moussa Ayyash. Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 17(4):2347–2376, 2015. Dostęp online: <https://doi.org/10.1109/COMST.2015.2444095>.
- [6] S. Chen et al. A review of environmental control strategies and models for greenhouse environments. *Sensors*, 25(5):1388, 2025. Dostęp online: <https://www.mdpi.com/1424-8220/25/5/1388>.
- [7] Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, 2010. Dostęp online: <https://www.control.utoronto.ca/people/profs/francis/dft.pdf>.
- [8] Docker documentation. Dostęp online: <https://docs.docker.com/>.
- [9] paho-mqtt. Dostęp online: <https://pypi.org/project/paho-mqtt/>.
- [10] Sqlalchemy. Dostęp online: <https://pypi.org/project/SQLAlchemy/>.
- [11] python-dotenv. Dostęp online: <https://pypi.org/project/python-dotenv/>.
- [12] Openai platform. Dostęp online: <https://platform.openai.com/docs/overview>.
- [13] Esp-idf programming guide. Dostęp online: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>.

- [14] Postgresql 18.1 documentation. Dostęp online: <https://www.postgresql.org/docs/current/index.html>.
- [15] Lorawan. Dostęp online: <https://pl.wikipedia.org/wiki/LoRaWAN>.
- [16] Dokumentacja języka python. Dostęp online: <https://docs.python.org/pl/3/reference/index.html>.
- [17] Polska dokumentacja c++. Dostęp online: <https://cpp0x.pl/dokumentacja/standard-C/1>.
- [18] Gemini api. Dostęp online: <https://ai.google.dev/gemini-api/docs?hl=pl>.

SPIS RYSUNKÓW

1.	Strona domowa systemu AgroSense	7
2.	Strona domowa systemu Farm Smart	7
3.	Strona domowa systemu Climate FieldView	8
4.	Diagram przypadków użycia systemu sterowania mikroklimatem szklarni	13
5.	Diagram architektury systemu (warstwa percepcji, sieci i aplikacji)	14
6.	Diagram wdrożenia systemu (ESP32 w szklarni, Raspberry Pi jako edge)	15
7.	DFD poziom 0 (kontekst) – główne podmioty i przepływy danych w systemie .	16
8.	DFD poziom 1 – szczegółowy przepływ danych: akwizycja, MQTT, przetwa- rzanie, zapis i sterowanie	17
9.	Diagram sekwencji: publikacja pomiaru i zapis danych w bazie	18
10.	Diagram sekwencji: decyzja sterowania i sprzężenie zwrotne	19
11.	Pętla sprzężenia zwrotnego – schemat ogólny z zakłóceniem d oraz szumem pomiarowym n	21
12.	Pętla sprzężenia zwrotnego – schemat rozszerzony z oznaczeniem sygnałów pośrednich x_1, x_2, x_3	21

SPIS TABEL

1.1	Porównanie analizowanych rozwiązań komercyjnych z projektowanym systemem.	8
3.1	Struktura tabeli <code>devices</code>	26
3.2	Struktura tabeli <code>measurements</code>	27
3.3	Struktura tabeli <code>control_actions</code>	27
3.4	Struktura tabeli <code>llm_analyses</code>	28
4.1	Zestaw przypadków testowych systemu	32

SPIS LISTINGÓW

3.1	Główna pętla odczytu i publikacji danych na ESP32	24
3.2	Uruchomienie usług infrastruktury (Docker Compose)	28
3.3	Przykładowa wiadomość telemetry MQTT (ESP32 → serwer)	29
3.4	Przykładowa wiadomość sterująca (serwer → element wykonawczy)	30

STRESZCZENIE

Praca przedstawia projekt i implementację inteligentnego systemu sterowania mikroklimatem szklarni, wykorzystującego technologie Internetu Rzeczy (IoT) oraz elementy sztucznej inteligencji. Celem pracy było stworzenie rozwiązania umożliwiającego autonomiczne monitorowanie i regulację kluczowych parametrów środowiskowych, takich jak temperatura, wilgotność powietrza i gleby oraz nasłonecznienie, w celu optymalizacji warunków wzrostu roślin oraz racjonalizacji zużycia zasobów.

System oparty jest na trójwarstwowej architekturze, obejmującej warstwę percepcji (mikrokontroler ESP32 z zestawem czujników), warstwę komunikacyjną (protokół MQTT w lokalnej sieci Wi-Fi) oraz warstwę aplikacyjną (serwer Raspberry Pi). Oprogramowanie serwerowe, zaimplementowane w języku Python, odpowiada za przetwarzanie danych, podejmowanie decyzji sterujących w oparciu o reguły progowe oraz integrację z modelem językowym (LLM) do zaawansowanej analizy danych. Istotną cechą rozwiązania jest możliwość pracy w trybie offline, uniezależniająca system od usług chmurowych, co zwiększa jego niezawodność i bezpieczeństwo danych. W pracy opisano proces projektowania architektury sprzętowej i programowej, implementację bazy danych PostgreSQL do archiwizacji pomiarów oraz wdrożenie systemu w środowisku kontenerowym Docker.

Słowa kluczowe: Internet Rzeczy, IoT, sterowanie mikroklimatem, szklarnia inteligentna, ESP32, Raspberry Pi, MQTT, sztuczna inteligencja, automatyzacja rolnictwa, system wbudowany.

ABSTRACT

This thesis presents the design and implementation of an intelligent greenhouse microclimate control system utilizing Internet of Things (IoT) technologies and artificial intelligence elements. The primary objective was to create a solution enabling autonomous monitoring and regulation of key environmental parameters, such as temperature, air and soil humidity, and solar irradiance, to optimize plant growth conditions and rationalize resource consumption.

The system relies on a three-layer architecture comprising a perception layer (ESP32 microcontroller with a sensor array), a communication layer (MQTT protocol over a local Wi-Fi network), and an application layer (Raspberry Pi server). The server software, implemented in Python, is responsible for data processing, making control decisions based on threshold rules, and integrating with a Large Language Model (LLM) for advanced data analysis. A significant feature of the solution is its capability to operate in offline mode, making the system independent of cloud services, thereby enhancing reliability and data security. The thesis describes the hardware and software architecture design process, the implementation of a PostgreSQL database for measurement archiving, and the system deployment within a Docker container environment.

Keywords: Internet of Things, IoT, microclimate control, smart greenhouse, ESP32, Raspberry Pi, MQTT, artificial intelligence, agricultural automation, embedded system.